

03 AWS Lambda with S3 Triggers - Lab Guide

AWS Lambda with S3 Triggers - Lab Guide

Learning Objectives

By the end of this lab, you will be able to:

- Create a basic AWS Lambda function
- Set up S3 triggers for Lambda functions (two different methods)
- Monitor Lambda execution using CloudWatch logs
- Deploy Lambda functions with and without dependencies
- Understand the Lambda execution environment

Prerequisites

- AWS Account with appropriate permissions
- Basic understanding of Python
- AWS CLI installed (optional but recommended)

Part 1: Creating Your First Lambda Function

Step 1: Create a Basic Lambda Function

1. Navigate to AWS Lambda Console

- Go to AWS Console → Services → Lambda
- Click "Create function"

2. Configure Function Settings

- Choose "Author from scratch"
- Function name: `s3-trigger-demo`
- Runtime: Python 3.9 (or latest available)
- Architecture: x86_64
- Click "Create function"

3. Add the Basic Code Replace the default code with:

```
import json

def lambda_handler(event, context):
    print("Event: ", event)
    print("Context: ", context)
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```

4. Deploy the Function

- Click "Deploy" to save your changes
- Note the function ARN for later use

Understanding the Code Structure

Event Parameter: Contains information about the triggering event (S3 object details, bucket name, etc.)

Context Parameter: Provides runtime information (remaining time, request ID, log group name)

Return Value: For API Gateway integration, should include statusCode and body

Part 2: Creating S3 Triggers (Method 1 - From Lambda Console)

Step 1: Add S3 Trigger from Lambda Page

1. Access Function Configuration

- In your Lambda function page, click on "Configuration" tab
- Select "Triggers" from the left sidebar

2. Add S3 Trigger

- Click "Add trigger"
- Select "S3" from the dropdown
- Choose your S3 bucket (or create a new one)
- Event type: Select "All object create events" or "s3:ObjectCreated:*"
- Prefix (optional): Leave blank for all objects
- Suffix (optional): `.txt` to trigger only for text files
- Click "Add"

3. Verify Trigger Setup

- You should see the S3 trigger listed in the triggers section
- Note the trigger configuration details

Part 3: Creating S3 Triggers (Method 2 - From S3 Console)

Step 1: Configure Trigger from S3 Bucket

1. Navigate to S3 Console

- Go to AWS Console → Services → S3
- Select your bucket

2. Set Up Event Notification

- Go to "Properties" tab
- Scroll down to "Event notifications"
- Click "Create event notification"

3. Configure Event Details

- Event name: `lambda-trigger-from-s3`
- Event types: Check "All object create events"

- Destination: Select "Lambda function"
- Lambda function: Choose your `s3-trigger-demo` function
- Click "Save changes"

Step 2: Grant S3 Permissions (if needed)

If you get permission errors:

```
aws lambda add-permission \
  --function-name s3-trigger-demo \
  --principal s3.amazonaws.com \
  --statement-id s3-trigger-permission \
  --action lambda:InvokeFunction \
  --source-arn arn:aws:s3:::your-bucket-name
```

Part 4: Testing and Monitoring with CloudWatch

Step 1: Test the Lambda Function

1. Upload a Test File

- Go to your S3 bucket
- Upload any file (e.g., a text file named `test.txt`)

2. Check Lambda Execution

- Return to Lambda console
- Go to "Monitor" tab
- Click "View CloudWatch logs"

Step 2: Analyze CloudWatch Logs

1. Access Log Groups

- You'll see a log group named `/aws/lambda/s3-trigger-demo`
- Click on the latest log stream

2. Review Log Output You should see output similar to:

```
Event: {
  'Records': [{
    'eventVersion': '2.1',
    'eventSource': 'aws:s3',
    'awsRegion': 'us-east-1',
    's3': {
      'bucket': {'name': 'your-bucket-name'},
      'object': {'key': 'test.txt', 'size': 123}
    }
  ]
}
Context: <__main__.LambdaContext object at 0x7f8b8c0a1f40>
```

Step 3: Enhance the Lambda Function for Better S3 Integration

Update your Lambda function code to extract S3 details:

```
import json

def lambda_handler(event, context):
    print("Full Event: ", json.dumps(event, indent=2))

    # Extract S3 information from the event
    for record in event['Records']:
        bucket_name = record['s3']['bucket']['name']
        object_key = record['s3']['object']['key']
        event_name = record['eventName']

        print(f"Bucket: {bucket_name}")
        print(f"Object: {object_key}")
        print(f"Event: {event_name}")

    return {
        'statusCode': 200,
        'body': json.dumps(f'Successfully processed {len(event["Records"])} record(s)')
    }
```

Part 5: Deploying Lambda with ZIP Files (No Dependencies)

Step 1: Create Local Function Files

1. Create a project directory

```
mkdir lambda-no-deps
cd lambda-no-deps
```

2. Create the Lambda function file (lambda_function.py):

```
import json
import datetime

def lambda_handler(event, context):
    current_time = datetime.datetime.now().isoformat()

    print(f"Function executed at: {current_time}")
    print("Event received:", json.dumps(event, indent=2))

    # Process S3 event
    processed_files = []
    for record in event.get('Records', []):
```

```

if 's3' in record:
    bucket = record['s3']['bucket']['name']
    key = record['s3']['object']['key']
    processed_files.append(f"{bucket}/{key}")

return {
    'statusCode': 200,
    'body': json.dumps({
        'message': 'Processing completed successfully',
        'timestamp': current_time,
        'processed_files': processed_files
    })
}

```

Step 2: Create and Upload ZIP Package

1. Create ZIP file

```
zip lambda-no-deps.zip lambda_function.py
```

2. Upload via AWS Console

- Go to your Lambda function
- In the "Code" tab, click "Upload from" → ".zip file"
- Select your `lambda-no-deps.zip` file
- Click "Save"

Part 6: Deploying Lambda with Dependencies

Step 1: Create Function with External Dependencies

1. Create project directory

```
mkdir lambda-with-deps
cd lambda-with-deps
```

2. Create requirements file (`requirements.txt`):

```
requests==2.31.0
boto3==1.34.0
```

3. Create the Lambda function (`lambda_function.py`):

```

import json
import requests
import boto3
from datetime import datetime

```

```

def lambda_handler(event, context):
    print("Lambda with dependencies executed!")

    # Use boto3 to interact with AWS services
    s3_client = boto3.client('s3')

    processed_objects = []

    for record in event.get('Records', []):
        if 's3' in record:
            bucket_name = record['s3']['bucket']['name']
            object_key = record['s3']['object']['key']

            try:
                # Get object metadata
                response = s3_client.head_object(Bucket=bucket_name,
Key=object_key)
                file_size = response['ContentLength']
                last_modified = response['LastModified'].isoformat()

                processed_objects.append({
                    'bucket': bucket_name,
                    'key': object_key,
                    'size': file_size,
                    'last_modified': last_modified
                })

                print(f"Processed: {bucket_name}/{object_key} ({file_size}
bytes)")

            except Exception as e:
                print(f"Error processing {bucket_name}/{object_key}: {str(e)}")

    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'Successfully processed objects with dependencies',
            'timestamp': datetime.now().isoformat(),
            'processed_objects': processed_objects
        })
    }

```

Step 2: Install Dependencies and Create Deployment Package

1. Install dependencies using --target flag

```

pip install -r requirements.txt --target .

```

Why use --target instead of regular pip install?

- `--target` . installs packages directly in the current directory
- Avoids installing in your global/virtual environment
- Ensures only the packages needed for Lambda are included
- Prevents conflicts with your local development environment
- Creates a clean, isolated deployment package

2. Create deployment ZIP

```
# Remove any __pycache__ directories
find . -name "__pycache__" -type d -exec rm -rf {} +

# Create the ZIP file
zip -r lambda-with-deps.zip . -x "*.pyc" "*/__pycache__/*"
```

Step 3: Upload and Configure**1. Upload the ZIP file**

- Follow the same upload process as before
- Select the `lambda-with-deps.zip` file

2. Increase timeout (if needed)

- Go to Configuration → General configuration
- Increase timeout to 30 seconds for functions with dependencies

Part 7: Testing Both Deployments**Test Scenario 1: Simple File Upload**

1. Upload a small text file to your S3 bucket
2. Check CloudWatch logs for both functions
3. Compare the output and execution time

Test Scenario 2: Multiple File Upload

1. Upload multiple files simultaneously
2. Observe how Lambda handles concurrent executions
3. Check for any errors or timeout issues

Part 8: Troubleshooting Common Issues**Issue 1: Permission Denied****Problem:** Lambda cannot access S3 bucket **Solution:**

- Check Lambda execution role permissions
- Ensure S3 bucket policy allows Lambda access

Issue 2: Function Timeout

Problem: Function times out with dependencies **Solution:**

- Increase timeout in Configuration
- Optimize code for better performance
- Consider using Lambda layers for large dependencies

Issue 3: Module Not Found

Problem: Dependencies not found in ZIP package **Solution:**

- Ensure dependencies are installed in the project root
- Check ZIP file structure
- Verify Python version compatibility

Best Practices and Key Takeaways

Performance Optimization

- Keep deployment packages small
- Use Lambda layers for shared dependencies
- Implement connection pooling for database connections
- Use environment variables for configuration

Monitoring and Logging

- Use structured logging with JSON format
- Set up CloudWatch alarms for errors and duration
- Implement distributed tracing with X-Ray

Security Considerations

- Follow principle of least privilege for IAM roles
- Use environment variables for sensitive data
- Enable encryption for Lambda environment variables

Cost Optimization

- Right-size memory allocation
- Use provisioned concurrency judiciously
- Monitor and optimize cold start times

Lab Exercises

Exercise 1: Enhanced S3 Processing

Modify your Lambda function to:

- Read the content of uploaded text files
- Count the number of words
- Store the word count in DynamoDB

Exercise 2: Error Handling

Add comprehensive error handling to:

- Handle malformed S3 events

- Implement retry logic for failed operations
- Send notifications on critical errors

Exercise 3: Multi-Environment Deployment

Create deployment scripts for:

- Development environment
 - Staging environment
 - Production environment with different configurations
-

Conclusion

This lab covered the complete lifecycle of AWS Lambda functions with S3 triggers, from basic creation to advanced deployment scenarios. You learned how to:

- Create and configure Lambda functions
- Set up S3 triggers using two different methods
- Monitor execution with CloudWatch
- Deploy functions with and without dependencies
- Handle common issues and implement best practices

The skills developed in this lab form the foundation for building event-driven, serverless architectures on AWS.